

PYTHON
PROJET JEU

Rapport Projet BluePrince

Étudiant :
CARRERA Hanus Alex
21211797

M1 ROB
Projet Jeu

16 novembre 2025

Résumé

Voici un rapport concis justifiant les choix d'implémentation

1 Introduction et contexte

Le but de ce jeu est de recréer le jeu *BluePrince* en 2D, simplifié, en Python en utilisant la programmation orientée objet.

Notre projet implémente une architecture modulaire . Le code est structuré en un ensemble de classes spécialisées, chacune responsable d'un sous-système précis.

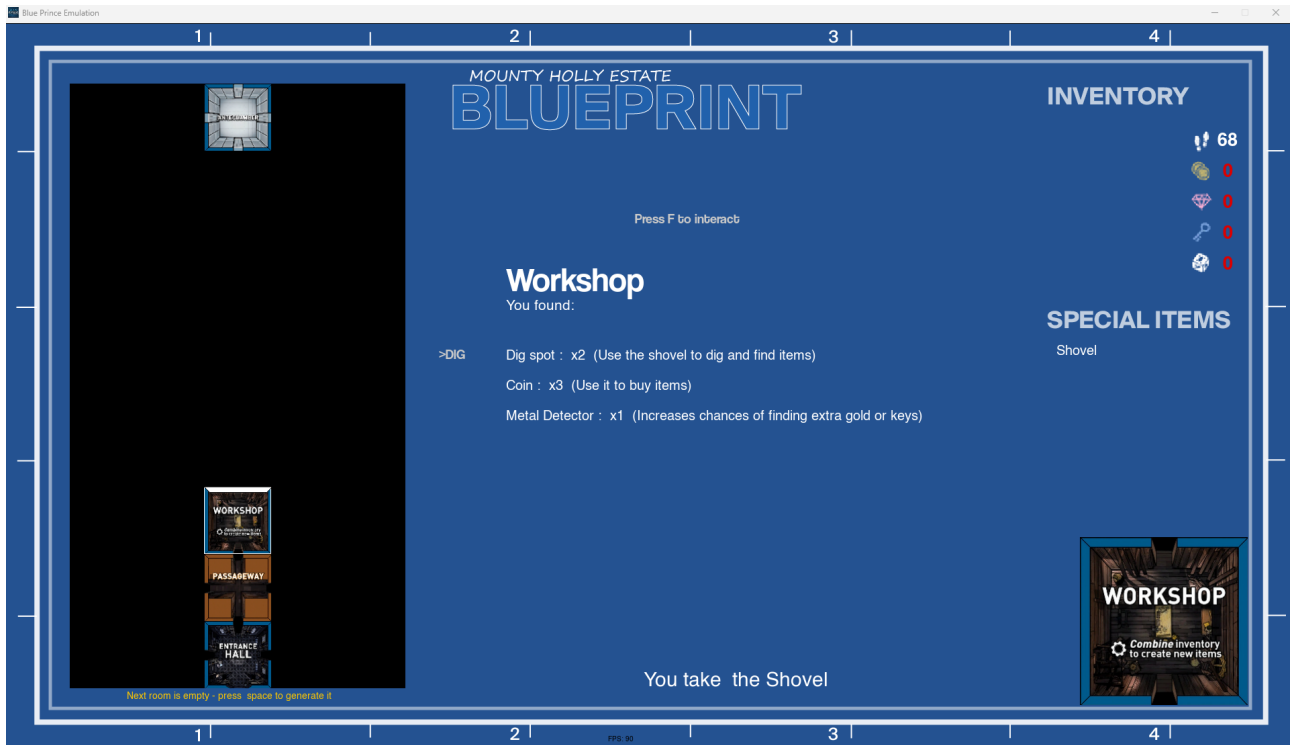


FIGURE 1 – Capture d'écran du jeu

2 Architecture Générale

Comme on peut l'observer dans le diagramme UML, le tout est construit autour de la classe abstraite *BaseHandler* et la classe *HandleData*. Voyons pourquoi.



Chaque enfant aura en référence l'objet de type *HandleData*, que nous verrons par la suite. Ceci simplifie les implémentations.

Cela permet de passer uniquement *HandleData* à chaque fois et garder un accès *global*. Il est donc plus facile d'implémenter des fonctionnalités intra-classes. Ceci rend cependant, le diagramme UML

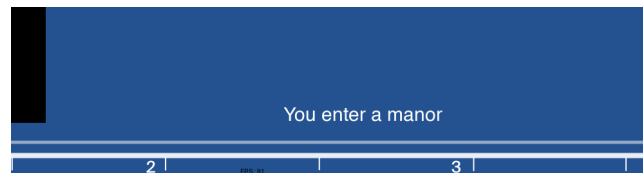


FIGURE 3 – Exemple historique

bien laid et difficile à lire. Pour toutes les références à *HandleData*, nous utilisons la couleur bleue dans le diagramme.

Elle inclut des méthodes utiles pour la mise à jour du texte de l'historique. Étant un *handler* comme les autres, elle hérite également de la classe *BaseHandler*. Sa méthode *update* se charge de vérifier si le jeu est fini ou pas..

2.3 Inventory

Il existe une classe générale *Inventory* qui permet de stocker les objets de type *item* (consommables ou permanents) et les objets de type *action*. *Action* est tout ce qui n'est pas *item* et avec quoi il est possible d'interagir (coffre, colis, endroit où creuser...). Plusieurs méthodes existent qui permettent de :

- Ajouter des objets à l'inventaire
- Retirer des objets de l'inventaire
- Gérer effets des consommables (manger = pas)

La classe *Inventory* inclut également *data*. Elle l'utilise pour mettre à jour le texte de l'historique quand le joueur interagit avec l'inventaire par exemple.

RoomInventory : deuxième héritage

Chaque chambre a également un inventaire propre. Cela permet d'ajouter facilement ou enlever des objets. *RoomInventory* hérite de *Inventory* et ajoute quelques méthodes très utiles pour la gestion des actions. Ce dernier héritage est très utile puisque la logique de l'inventaire *Player* et *Room* est la même. La seule chose qui change est le *sens* des *transactions* : quand le *Player* ajoute un objet dans son inventaire, suit retirer le même objet de l'inventaire de la chambre...

3 Utilisation JSON

Afin d'éviter d'avoir des dictionnaires extrêmement longs et de pouvoir avoir un fichier général avec toutes les données relatives aux chambres, nous avons décidé d'utiliser des fichiers *.json*.

```
{
  "Greenhouse": {
    "image": "Greenhouse.jpg",
    "color": "green",
    "rarity": 1,
    "cost": 1,
    "doors": [
      "N",
      "E",
      "W",
      "S"
    ],
    "placement_condition": 2,
    "q": 1,
    "items": {},
    "possible_items": {
      "metal_detector": [1,0],
      "shovel": [1,0],
      "hammer": [1,0],
      "dig_spots": [4,1]
    },
    "description": "You are more likely to draw Green Rooms while drafting"
  },
}
```

FIGURE 4 – Extrait du JSON

Toutes les données pertinentes y sont listées, telles que : les portes présentes, les objets forcément présents, les objets (*items* et/ou *actions*) possiblement présents, le coût (en gemmes) de la chambre, une description textuelle pour l'interface, la quantité *q* dans la pioche (si nous voulons avoir une répétition), la couleur, le *dir* relatif de l'image, la rareté et finalement la condition de placement.

Une autre raison pour laquelle nous avons utilisé le fichier json est la suivante :

Nous avons automatisé la conversion de *Exemple_icon.webp* à *Exemple.jpg*. Le script élimine le *_icon*, place l'image dans le fichier correspondant, crée l'entrée Exemple dans le *json*. Ainsi, il suffit remplir les informations à chaque fois uniquement, au lieu de manuellement copier le nom (erreurs possibles).

```
# --- Update JSON if missing ---
if base_name not in room_data:
    room_data[base_name] = {
        "image": jpg_filename,
        "color": "bleue",
        "rarity": 0,
        "cost": 0,
        "doors": ["N", "E", "W", "S"],
        "placement_condition": 0,
        "q": 1,
        "items": {},
        "possible_items": {},
        "description": ""
    }
```

FIGURE 5 – Initialisation du json

Le même principe est utilisé pour les fichiers *.mp3*. Un autre script lit tous les fichiers dans le fichier *sfx* et les ajoute au json *sfx*.

4 Params

Parlons maintenant de *params*, qui n'est pas inclus dans le diagramme UML puisqu'il ne s'agit pas d'une classe. Nous utilisons ce fichier pour stocker en mémoire globale et accessible les données statiques. Cela permet d'avoir dans un emplacement tous les paramètres, et de faire des modifications rapidement. S'agissant de données statiques (origines text, taille, police...), il n'y a aucune raison pour laquelle il serait justifier d'utiliser une classe.

```
ITEMS_DESCRIPTION_DICT = {
    # nom interne: [nom displayed, phrase descriptive displayed, type tempo/permanent ou action, rareté, verbe si ajoute par utilisateur, verbe si utilisé par utilisateur]
    "gold": ["Coin", "Use it to buy items", "t", 1, "TAKE", "USE"],
    "gems": ["Gem", "Use it to buy special rooms", "t", 2, "TAKE", "USE"],
    "rabbits_foot": ["Rabbit's foot", "Greater chance of finding items", "p", 3, "TAKE", "NAN"],
    "keys": ["Key", "Useful for opening doors and chests", "t", 1, "TAKE", "USE"],
    "apple": ["Apple", "Restores 2 steps", "a", 2, "TAKE", "BUY"],
    "banana": ["Banana", "Restores 3 steps", "t", 2, "TAKE", "BUY"],
    "lockpick": ["Lockpick", "You can now use the lockpick to open doors", "p", 3, "TAKE", "USE"],
    "metal_detector": ["Metal Detector", "Increases chances of finding extra gold or keys", "p", 3, "TAKE", "USE"],
    "dig_spots": ["Dig spot", "Use the shovel to dig and find items", "a", 2, "DIG", "DIG"],
    "trunk": ["Trunk", "Use a key or the hammer to open and find items", "a", 3, "OPEN", "NAN"],
    "shovel": ["Shovel", "You will need this to dig", "p", 3, "TAKE", "USE"],
    "dice": ["Dice", "Use this to redraw rooms when drafting", "t", 2, "TAKE", "USE"],
    "steps_left": ["Step", "", "t", 0, "GAIN", "USE"],
    "hammer": ["Hammer", "Use this to break open chests without keys", "p", 3, "TAKE", "USE"],
    "package": ["Package", "A mysterious package. Who knows what is inside", "a", 1, "OPEN", "USE"],
    "apple_buy": ["Apple", "Buy an apple - 2 coins", "a", 0, "BUY", "NAN"],
    "banana_buy": ["Banana", "Buy a banana - 3 coins", "a", 1, "BUY", "NAN"],
    "cake_buy": ["Cake", "Buy a cake - 6 coins", "a", 2, "BUY", "NAN"],
    "sandwich_buy": ["Sandwich", "Buy a sandwich - 8 coins", "a", 3, "BUY", "NAN"],
    "meal_buy": ["Meal", "Buy a meal - 15 coins", "a", 3, "BUY", "NAN"],
    "locker": ["Locker", "Use a key to open a locker", "a", 1, "OPEN", "OPEN"],
}
```

FIGURE 6 – Dict Descriptif des Objets

Cela permet d'améliorer la performance du code et facilite le développement (sinon nous aurions un *handleData* gigantesque et le diagramme UML serait d'autant moins lisible).

Dans le fichier *params*, nous avons plusieurs dictionnaires qui stockent des données relatives aux *items/actions*. Ces données sont du type : nom d'usage, nom pour interface, descriptif pour interface, type (*action/item permanent/item tempo.*), rareté associée, verbe pour quand le joueur l'ajoute à son inventaire, verbe pour quand il le retire.

Nous avons également un *dict* avec les données associées aux consommables, telle que le coût et le nombre de pas que l'objet apporte.

```
POSSIBLE_CONSUMABLES_DICT = {
    "apple": {"gives":2,"costs":2},
    "banana": {"gives":3,"costs":3},
    "cake": {"gives":10,"costs":6},
    "sandwich": {"gives":15,"costs":8},
    "meal": {"costs":15,"gives":20},
}
```

FIGURE 7 – Dict Descriptif des consommables

Finalement, les différentes *actions* ont également un *dict* associé :

```
POSSIBLE_ITEMS_PER_ACTION_DICT = {
    "dig_spots": {"gold":[5,1],"keys":[3,1],"gems":[2,1]},
    "package": {"gold":[10,1],"keys":[2,1],"gems":[1,1],"rabbits_foot":[1,1]},
    "trunk": {"keys":[2,1], "gems":[2,1], "gold":[3,0], "dice":[2,0],"lockpick":[1,0]},
    "locker": {"gems":[1,0],"gold":[5,1],"apple":[1,0],"keys":[1,0]}
}
```

FIGURE 8 – Dict Descriptif des actions

Chaque *action* listée ici est une *boîte* qui peut contenir différents objets. La valeur associée à chaque clef est une liste de type $[q,1]$. Le q représente la quantité maximale trouvable et le 1 peut être égal à 0 ou 1 . Quand il est égal à 0 , alors la valeur trouvée est celle passée en argument. Sinon, il est possible de trouver une quantité entre 1 et q .

Il serait pertinent de mentionner une centaine d'implémentations additionnelles, mais ce rapport n'est pas censé toutes les mentionner.

5 Conclusion

En conclusion, grâce à une approche modulaire, centrée autour de *handlers* et de *data*, nous arrivons à voir un code qui est simple à débiter, améliorer et tester. Le choix de conserver des données statiques dans *params* rend les petites modifications simples et améliore la performance du code. Finalement, l'utilisation de *json*/dictionnaires s'est avérée être extrêmement utile.